

Inigo Quilez :: articles :: rational rendering and floating bar - 2018



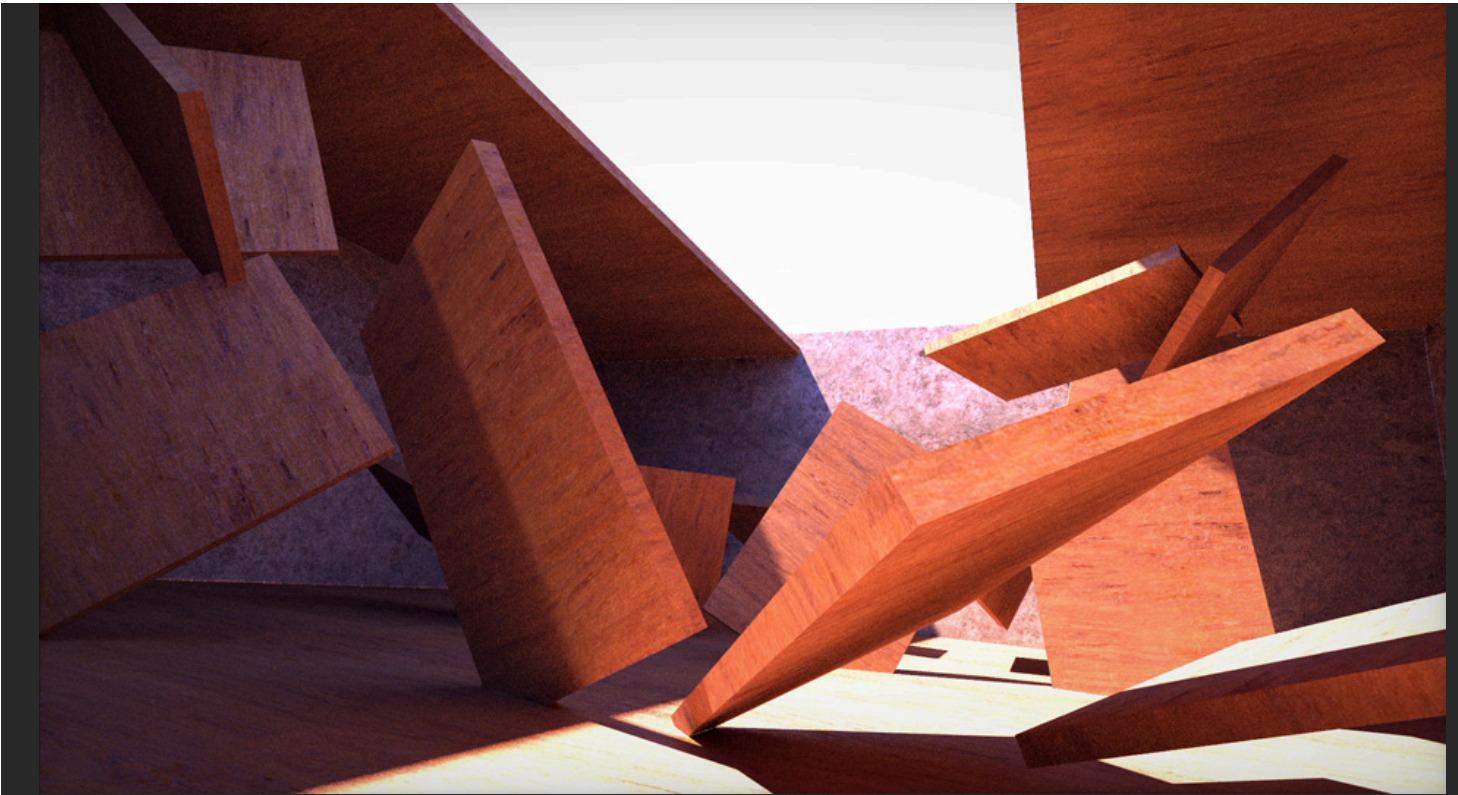
Intro

"So, what would happen if we replaced the computer's default floating point numbers with an alternative system made of only rational numbers, and tried to render an image with those?"

I posed myself this question after reading a tweet by Morgan McGuire, a well known computer graphics researcher and educator. He was reflecting on how computer science students often get surprised when they learn for the first time that the floating point numbers in our current computers have important precision tradeoffs that make simple certain things impossible, such as checking whether four points are coplanar. The theory of algebra says that one can check for such coplanarity by computing a determinant, or equivalently, doing some cross and dot products. If the result of that operation is zero, then the four points are coplanar, and if the result is not zero, they aren't. The problem of course is that the default floating point arithmetic computers employ work in finite precision, and so decimals are being discarded constantly. This doesn't give us much headaches most of the times, but some other times it does - in the case of the coplanarity test we are discussing, the continuous truncation of floating point operations in each of the determinant's or cross/dot products inner multiplications and additions, the results will never be exact and a zero exactly even when the four points were mathematically coplanar.

This is common knowledge to any computer graphics aficionado like myself, but it made me thing - if the coordinates of the four points were given as rational numbers (that is, a fraction, or the the ratio of two integers) and all operations were carried as rational computations using the regular rules for addition and multiplication of fractions), the final result should be a rational number and the numerator should be exactly zero, and solve our problem!

Furthermore, if we had a 3D renderer and fed it with a scene where all input data (vertex coordinates, 3D transformations, etc) were given as rational numbers instead of floating point numbers, then all operations from ray generation, to acceleration structure traversal, to the ray-triangle intersection could be carried in rational numbers too, to eventually produce intersection points expressed as rational coordinates, and have a "perfect" rendering.



Pathtracing using rational numbers in the "floating **bar**" format invented for this article, instead of the usual floating **point** numbers

Okey, before we move on, let's do a quick refresher for those who don't remember rational numbers very well. As said, a rational number is one that can be expressed as the ratio of two integers, such as $1/2$ or $355/113$. Rational numbers can be added, subtracted, multiplied and divided to produce rational numbers again. Formally, one says that a set of rational numbers forms a field, and is closed under the four basic arithmetic operations, which means that rational numbers combine to produce rational numbers only and not something else that cannot be expressed as the ratio of two integers. In other words, rational numbers will never "leak" into the territory of infinite decimals, imaginary numbers, or other inconvenient things. These are by the way the numbers Pythagoras found pure and perfect.

Now, back to rendering - the typical rendering operations such as ray generation, bounding box tests during BVH traversal, ray-triangle intersections, ray reflections for shading, etc, are all based on cross and dot products and scalar multiplications, including all coordinate transformations and matrix inverses and quaternion logic. Cross and dot products in turn are based on the four basic arithmetic operations of addition, subtraction, multiplication and division. So that means that in a world of rational arithmetic, cross and dot products will also be computed exactly without truncations, and so will be ray generation, bounding box tests during BVH traversal, ray-triangle intersections, ray reflections for shading, etc! The first problem I could anticipate at this point was the size required to store these rational numbers, but I was familiar with BIGNUM libraries, and from a theoretical standpoint anyways the idea of producing mathematically exact renders

seemed sound in principle.

Or almost sound; the second problem I could foresee was that there are a few operations that appear in rendering tasks that do fall outside of the rational field. These are the trigonometric functions (sin, cos, atan) and also the square root. For the former, I've always expressed my opinion that trigonometry should only be employed in the outermost layers of an engine, close to the user's input, and not in the engine itself - if you find yourself doing trigonometry in the inner parts of your renderer, you are most probably doing things wrong (and have shown [how to fix one of the most typical examples](#)). In short, it is very easy to build a trigonometry-free engine, which is faster and more elegant. So I was not expecting any issues here..

As for square roots (needed in vector normalization among other operations), the problem is that rational numbers are not closed under rooting, that is, the square root of a rational number is seldom a rational number. So using them ruins our chances of having an exact renderer. However, except for intersecting and computing bounding boxes of conics sections (ellipsoids, cylinders, etc) there's actually no need for ray normalization or square roots in a renderer. This might surprise some readers, I feel the computer graphic community has fallen into the habit of normalizing every single vector that crosses their path, by default, not knowing whether they should do it or not, but doing it anyways "just in case". In reality, it's very rarely necessary. Ray generation certainly doesn't need it - the ray doesn't need to be normalized in order to perform BVH traversal or ray-triangle intersections. Reflections also don't need it either. I can barely think of any geometrical operation that needs normalized vectors. Only during shading/brdf-ing/coloring one needs to normalize rays and surface normals. But at this point we are outside the realm of geometry and exact surface intersections, coplanarity tests etc, so I'd personally claim success if I made a renderer where square roots are invoked only at surfacing/coloring time.

So, to summarize, at least during this quick mental exercise ("shower thining") I was confident I could write a renderer without square roots or trigonometry calls for all geometry matters, that runs only in rational numbers and produces exact visibility and geometrical results including unambiguous coplanarity tests on top of it if so desired.

So this article is the story of the two nights exploration that followed that though and I conducted to evaluate how feasible it was, including the multiple things that I learnt and invented along the way, and the few surprises I found. I wrote the article following the developments and side quests as they unfolded, rather than in my usual structured and formal way. Lastly, you could think the image above is a bit of a spoiler, but read until the end of the article because I have you both good and bad news.

Setup

To start the exploration, the first thing I did was to implement a bare minimal tracer in [Shadertoy](#) with a super simple scene composed of a plane, a sphere, a box and a triangle - the building blocks of a real renderer. This was of course using regular floating point arithmetic. Then I copy+pasted the code to a C++ file and after a couple minor tweaks to convert from GLSL to C++, I was up and running soon enough. That gave me my traced image rendered in the CPU with regular IEEE754 floating point numbers, which I'd use for reference. For this version I removed all ray and vector normalizations in the renderer, which required square roots, since none of them were needed as discussed earlier. While later on we'll see that we can of course still do square roots, I wanted to keep the code as mathematically pure as possible to see how far could I go with exact, rounding-free arithmetic of rational numbers.

The last preparation step was to take all the vec3, mat4x4 and other basic algebra/math classes and modify them to use "rational" numbers instead of "float". I wrote a little "rational" class with a sign bit, a numerator and a denominator unsigned integers, and then overloaded all common math operators (add, sub, mul, div, negation, comparisons, etc). The change was quick and went smoothly. I also implemented the rest of the usual operations quickly (abs, sign, mod, fract, floor, etc). In theory, this was all I really needed to do in order to get pretty rational renders.

```
// a 65 bit rational number
struct rational
{
    bool    s; // sign
    uint32_t n; // numerator
    uint32_t d; // denominator
};

bool operator==( rational const & a, rational const & b );
bool operator< ( rational const & a, rational const & b );
bool operator> ( rational const & a, rational const & b );
bool operator< ( rational const & a, int const & b );
bool operator> ( rational const & a, int const & b );
rational operator*( rational const & a, rational const & b );
rational operator+( rational const & a, rational const & b );
rational operator/( rational const & a, rational const & b );
rational operator-( rational const & a, rational const & b );
rational floor( const rational & a );
rational fract( const rational & a );
rational min( const rational & a, const rational & b );
rational max( const rational & a, const rational & b );
rational abs( const rational & a );
rational sqrt( const rational & a ); // not exact
```

Test 1 - the naive thing

But let's see how this initial implementation was in more detail. You must know that when I'm exploring unknown territory, I always try the simplest thing first and see how that goes before going fancy or sophisticated. And the simplest way to implement rationals is to use two integers, a numerator **N** and a denominator **D** that produce a value $x = N/D$, for a total of 64 bits of data. I chose to enforce both numerator and denominator to be positive, and I stored the sign of the number in a one extra separated bit, to simplify things and remove some ambiguities. Let's see what the four basic arithmetic operations look like in this rational form.

First, note the subtraction of $a - b$ is just the addition of a and the additive inverse of b , ie, $a + (-b)$, where $-b$ can be computed simply by flipping the sign bit of b . Similarly, performing the division a/b is the same as doing the multiplication of a and the (multiplicative) inverse of b . Or in other words, $a/b = a \cdot (1/b)$, where $(1/b)$ can be computed by simply swapping the numerator b_n and denominator b_d of b . So, this is the first interesting property of rational arithmetic - divisions and multiplications have the same cost, so unlike in regular floating point rendering where division are usually avoided or delayed or hidden under the latency of slow texture fetches, in rational arithmetic there's no need to fear them.

Ok, so it's clear we just need to implement addition and multiplication and division and subtraction will follow from there:

$$\begin{aligned}
 a + b &= \frac{a_n}{a_d} + \frac{b_n}{b_d} = \frac{a_n \cdot b_d + a_d \cdot b_n}{a_d \cdot b_d} \\
 a \cdot b &= \frac{a_n}{a_d} \cdot \frac{b_n}{b_d} = \frac{a_n \cdot b_n}{a_d \cdot b_d} \\
 a - b &= a + (-b) \\
 a/b &= a \cdot (1/b)
 \end{aligned}$$

The first thing to note is that addition is very expensive, and can easily overflow the word size (32 bits in this case). Multiplication also will generally overflow. Some other things to note - the propagation of the sign for the multiplication is just an xor, since two positives do a positive and so do two negatives. The propagation of the sign for the addition is more tricky and I implemented it quickly with three branches (the addition is trivial if both signs of a and b agree, but when they don't you have to pick the smallest of the two and subtract it from

the other - I won't bother with the small implementation details here anymore, I'll put my source code somewhere).

I will also skip the implementation of the `fract()` and `floor()`, which you'll find easy and beautiful if you try on your own. The comparison operators deserve some care as well - once signs have been taken care of and assuming **a** and **b** are positive, then

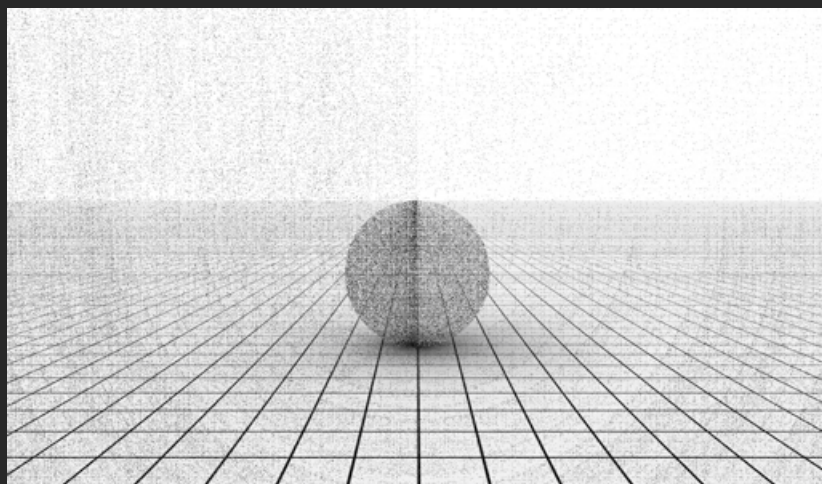
$$\frac{a_n}{a_d} > \frac{b_n}{b_d} \leftrightarrow a_n \cdot b_d > a_d \cdot b_n$$

The important thing to note here is that even comparison require us to do a couple of multiplications and potentially need to promote things to the next word size, and this will become important in a bit later in this article.

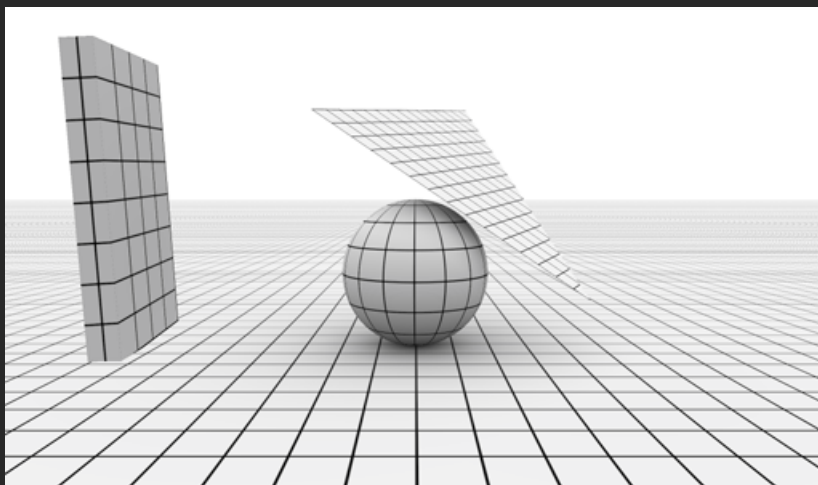
Lastly, we'll consider square roots later in its own dedicated section as well, but suffice to say that for now I approximated the square root by taking integer square roots of both numerator and denominator. This is a really bad approach, but sufficient for now, given that in this render I only need them for the sphere in the scene.

So, with all this code in place, a rather naive implementation if you ask me, I was ready to launch my first render of the plane+sphere+triangle+box, and see what the heck I would get on the other side.

Well, the left image is what I got with this naive implementation which uses 65 bit rational numbers, and on the right the reference image with regular floating point numbers (32 bit):



Naive 65 bit rationals



Floating point reference

The result was pretty bad; the box and triangle didn't even show up in the render, and the sphere and floor plane were very noisy. I was pretty sure these problems were not bugs or coding errors, but a direct consequence of my super naive implementation of rational numbers and arithmetic.

It didn't take long to identify the problem - with each rendering operation (dot product, vector addition, etc, the numerator and denominator of my rational numbers were getting bigger and bigger without control. Remember that all operations, including rational addition and subtraction, do require multiplication of integers. And multiplication of integers can only grow and grow. So imagine the following: let's assume our original world units were meters, and that we were snapping our source geometry (vertices and camera) to a millimeter accuracy, which is a sensible accuracy. Then, the source data for the scene of 1 kilometer in size, the coordinates of our world require 20 bits of usage already. For a small scene of 10 meters, that would be 18 bits. At the same time, for a typical HD screen with 4X4 supersampling, the ray direction rationals would need 13 bits (1920x4 unique directions horizontally). So, as soon as the first ray and geometry interaction happened, that is, as soon as the very first arithmetic instruction that combined our ray and a camera or object transform, even if that was only adding coordinates, the numbers involved would become $18 + 13 = 31$ bits long. This is close to overflowing my 32 bit numerator and denominator! In other words, even before I had completed my very first dot or cross product at all, I was already at the verge of catastrophic overflow errors. It could then only expect that by the time the cross product was completed, the renderer would already be operating on completely wrong information. Of course, not all pixels were that bad - for rays near the center of the screen and geometry near the origin of the scene, where numbers stay small, the overflows were much delayed or sometimes totally avoided (the floor plane and bits of the sphere). But in general, it was not surprising that I was barely seeing anything in my renders.

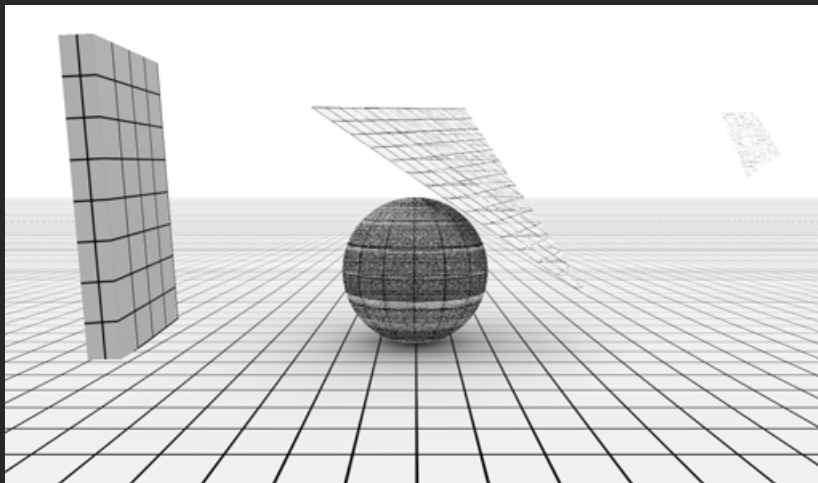
Test 2 - GCD reduction

I thought one solution would be to replace my 32 bit numerator and denominators with some infinite precision numbers, like in a BIGNUM library. But before requesting unbounded amounts of memory from the system, I tried a different approach based on the following observation: different rational numbers can represent the same quantity. For example, $6/12$ is the same value as $1/2$, but the former utilizes $3+4=7$ bits while the later only needs $1+2=3$ bits. So, my idea was this - if after (or before) each basic arithmetic operation, right in the operator overloads themselves, I extracted all the common factors from the numerator and denominator of the current number and reduced the fraction to its simplest form (divide by all common factors), then maybe I could keep all quantities under control without ever growing. That would allow me to delay overflows and keep operating with exact arithmetic (and without losing any precision) for a longer time. Maybe long enough to get clean rendered images?

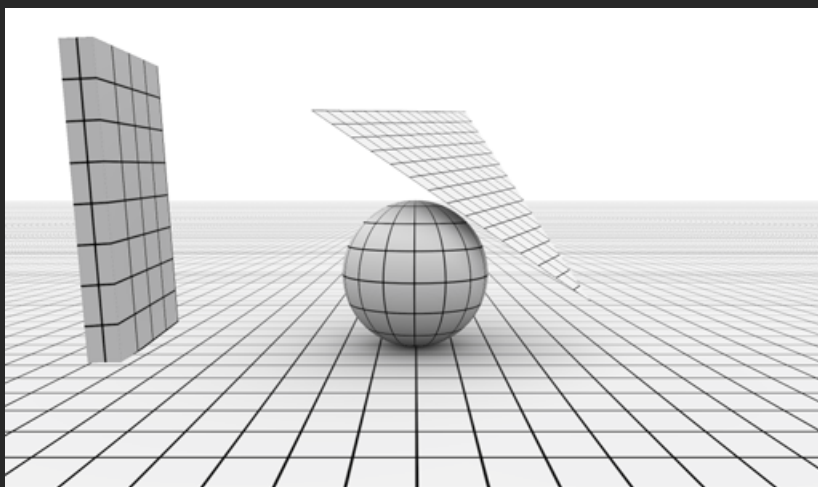
So, mathematics refresher again - say, we know we can take $588/910$ and simplify it to $42/65$, since the 14 divides both 588 and 910. The number 2 also divides 588 and 910, but we want to find the largest number that divides both, so we can reduce the fraction to its smallest form so we can save the maximum possible number of bits possible. Finding the largest number that divides two other numbers simultaneously can be done with the Great Common Divisor (GCD) algorithm, of which there are efficient forms everywhere. In my case I copied directly from the Wikipedia article on the GCD, and sped it up a little by doing the bit scan step using x64 intrinsics.

This was pretty quick to get running, so now armed with the GCD, I extended my "rational" class to constantly keep simplifying my fractions as they were being operated on, that is, inside the implementation of the add and multiply operators as I said. Now, there are two ways I could have done that:

This first one was to promote the intermediary result of the addition and multiplication operators to the next worth length (`uin64_t` in the case of the current 32 bit numerator/denominator approach), perform the GCD in that wider data type, and then demote the result to the source bit length (32 bits). The second approach was to analyze how a_n , a_d , b_n and b_d combine with each other in both arithmetic operators, and extract common factors across them **before** performing the multiplications. This second approach prevents in principle the need for promotion to 64 bit arithmetic, but it's more work. So, knowing that I wanted to move fast now and that I could always go back to this issue later and implement it the smart way, I chose to implement it the simple way and keep moving fast (the night is short). So, with the GCD implemented in the add and multiply operators, let's see what kind of render I was able to produce:



65 bit rationals with GCD reduction



Floating point reference

Much better! Still not there, but this was now much more promising. I got my box and triangle to finally show up, and the sphere felt more solid now. There was still some funny artifact on the top right of the image though, and the rationals still overflowed for many of the pixels, producing lots of image acne. However, it's worth realizing that for some large amount of pixels, I was getting **exact** and perfect results! Meaning, the tracer found the mathematically exact intersection points, which was kind of the point of trying rational numbers in the first place!

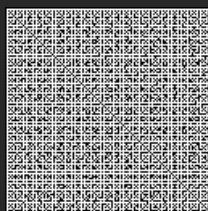
Before going on my next step in the quest of making rational numbers actually viable for rendering, I want to take a break and do a little detour. Because even though the GCD had helped a lot, I still felt I was wasting bit-space. After all, the GCD was taking care of numbers such as $1000/2000$ and reducing them to $1/2$. But that means, that from all possible combinations of the 65 bits in my rational number representation, many were "invalid" or "wasted" combinations, such as the $1000/2000$ bit pattern indeed. My intuition was that **most** of the space must have been wasted. But was I right?

Detour - on coprime numbers

Below are two pictures. The first one shows the bit usage for rational numbers of 10 bits, 5 bit numerator and 5 bit denominator, which is why it is a 5x5 pixel image. The second image shows the bit usage for rationals of 14 bits, 7 for numerator and 7 for denominator, and it's therefore a 7x7 pixel image. The horizontal and vertical axis of the graphs represent all bit combinations that numerators and denominators can take. A black pixel means that particular numerator and denominator bit combination is never used, because it can be simplified to a simpler one. A white pixel indicates a valid bit combination (a fraction that cannot be simplified). For example, the diagonal is all black except for the 1/1 pixel, since all fractions of the form n/n can be reduced to 1/1. The pixel 4/2 is also black, but 13/7 is white.



bit usage for 5/5 rationals



bit usage for 7/7 rationals

My intuition was telling me that these images would be mostly white, meaning that most bit combinations wouldn't be used. However, to my surprise at least, the images had a lot of white in them. I did some pixel counting, and I quickly realized that the proportion of useful pixels converges to 60.8% as I increased the number of bits in numerator and denominator. A bit of research online taught me that this proportion happens to actually be **exactly** $6/\pi^2$, which is the probability of two random numbers to be coprime (to not have common factors). Wait, why is PI showing up to this party, you might ask? Well, "Six over Pi squared" happens to be the value of one over the **Riemann Zeta function** evaluated at 2, $1/\zeta(2)$. This might be not completely surprising since the Riemann Zeta function frequency shows up in problems where there are prime and coprime numbers involved.

In any case, it seemed I was wasting *only* about 40% of the bit combinations in my rational representation. On one hand, this felt like a lot of waste... But on the other hand, I could also see it as wasting only one bit (a bit less in fact). One bit out of 65 didn't feel like a show-stopper to me. So with that in mind, I decided to move forward with my explorations.

And so I did some rapid learning on Stern-Brocot and Calkin-Wilf trees, which would have allowed me to make full use of all the available bits, but the range of values I could get with them is really tiny, so I discarded the idea quickly and moved on. I think at this point I should formally credit [Wikipedia](#) for being a constant source of learning.

Alright, so let's recap where I was at this point in the night: I could render some broken images but there was some hope. But my main problem was that I was at the mercy of the incursion of prime numbers in the data or the computations. Indeed the prime numbers controlled when the GCD algorithm could or could not simplify fractions. Specially large primes were very dangerous - as soon as one made it into either the numerator or denominator, it would stay in there, fossilized forever, "polluting" the rational number and making it larger, with little hopes of appearing again in the denominator or numerator for the GCD to simplify. Statistically speaking, I felt my rationals were guaranteed to eventually explode; it was just a matter of how early or how late.

To make matters worse, and to tackle the elephant in the room, I was using 65 bits for my rationals (1+32+32). This is twice as many bits as a standard floating point number, and still getting rather broken renders. I of course tried to use 1+15+16 rationals and check what quality I was getting for a "reasonable" data size, but the current numerator+denominator+GCD approach produced images that were just unrecognizable.

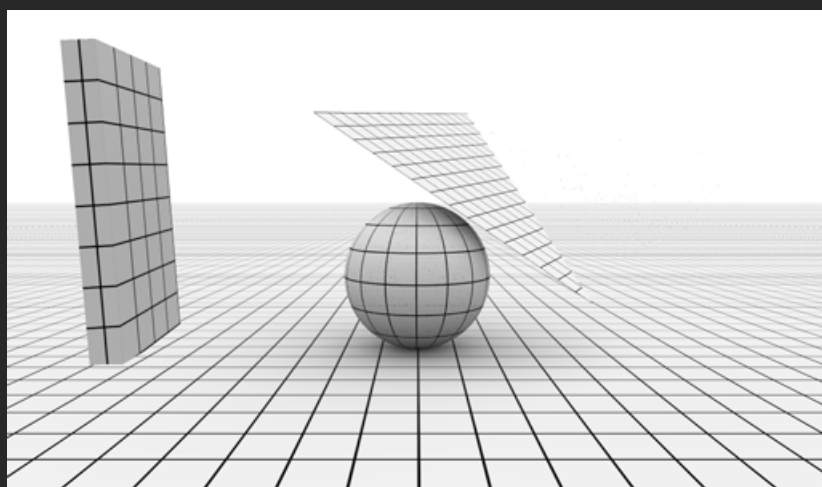
Test 3 - rational normalization

So I was using twice the memory of a regular renderer, and was getting worse results. So I was at a dead-end here, and I needed to do something dramatic. It seemed like I needed to start chopping numbers and losing precision if I wanted to prevent the numbers from growing without limits. But the whole exercise started from the idea of exploring **exact** rendering, so chopping wasn't in the menu. And that's why, at this point, I realized I had to give up on the initial idea of perfect resolution (without BIGNUM unbound resources), and keep exploring new territories, if only for fun, and see where I would land. This wasn't a new thing for me, I often have an idea which has the only purpose to kick off some research process that will land in some unexpected place, not to determine where it should land. Or as John Cleese once said, wrong ideas can lead you to good ideas, the creative process does not need to be a sequence or progression of logically correct steps at all times).

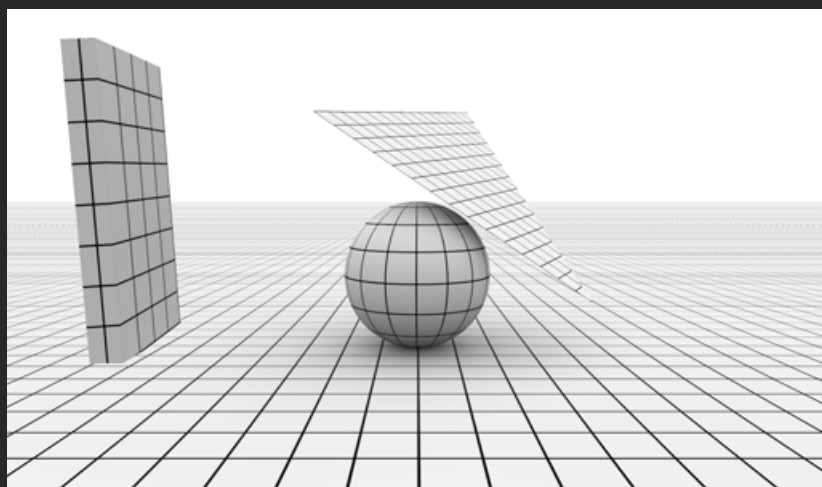
Anyways, I decided to see what I'd happen to the renders if I somehow managed to prevent numerator and denominator from overflowing. The simplest thing to do was to shift both numerator and denominator enough bits to the right, as necessary when necessary, until they fitted in the assigned bit space. This effectively does an integer division by a power of two both in numerator and denominator, and hence the value of the rational

stays **approximately** unchanged, given that when this occurs the numerator or the denominator are large (sometimes both) since we are talking of a near-overflow situation.

So my first approach was super simple - when overflow was going to happen, I looked at the number of bits needed by the numerator and the denominator, took the maximum, and shifted both by that amount of bits (making sure I was rounding to the nearest integer when doing so). When this was implemented both in the addition and multiplication operators, things started to look visually reasonable:



65 bit rationals with GCD reduction and normalization



Floating point reference

Not bad. In fact, things were looking good enough that I finally let myself concern with the bit usage of my implementation. So instead of 65 bits, I tried 33 bits (1+16+16) again to see what I could get with a roughly float32 "equivalent" data size. And images surprisingly came pretty good this time! I could still see that some of the edges in the sphere had little holes and the texture pattern in the triangle had some tiny discontinuities. But it was kind of good, the rational normalization was definitely helping. That gave me energy to keep looking

for some new ideas.

Test 4 - floating bar

At this point I decided to switch gears and stop making excuses - now that I was in “inexact” territory, if I was going to find anything interesting around using rational numbers for rendering, it better be exactly 32 bits total and no more. I rather get some good idea or stop there and call it a night (well, two nights, since this was the beginning of my second session).

My first thought was that I could stick to the GCD and renormalizations ideas, but I needed to be smarter about the storage and bit utilization. The first thing that came to mind was that, even though numerator and denominator can get large, often they don't get large both at the same time. Oftentimes the denominator is small, so the numerator should be allowed to be large. And some other times it is the other way around. In other words, the unused bits of one of the two integers could be used by the other to express larger values as needed. Then I realized that the same way a floating point number is pretty much a fixed point format with the “fixed” point made variable (or “floating”), I could take my rational numbers and make the location of the division/fraction/bar variable as well. Meaning, instead of hardcoding it to be 16:16, I could let the same 32bit variable sometimes represent a 16:16 rational, but also some other times a 5:27 rational or a 13:9 one, as needed.

It was worth trying. A few lines of packing/unpacking code in the internal setters and getters for numerator and denominator of my “rational” class was fast to code anyways. I just had to reserve some bits to store the position of the rational bar (the bit separation between numerator and denominator). The most logical layout to try, I thought, was 115|26, meaning:

1 bit : sign

5 bits: bar/fraction position (B)

26 bits: numerator and denominator data merged, numerator high 26-B bits, denominator low B bits

where the bar position (B) determines the size of the denominator. For example, the number 7/3 would in principle be

$7/3 = 0\ 00010\ 000000000000000000000000111\ 11$

where the sign is 0 for “positive”, the bar is “2” to indicate the the denominator (the number 3) needs 2 bits to

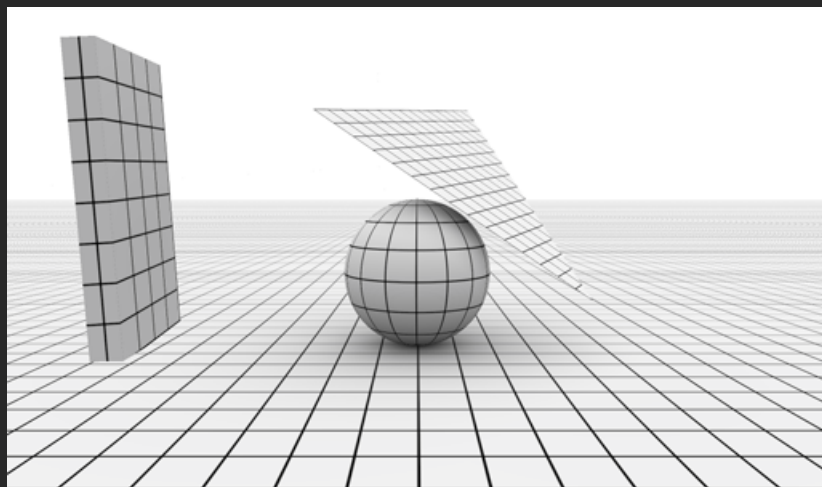
be represented and that the rest of the bits are left for the numerator.

Now, the readers that have worked with the IEEE754 standard might find the following observation familiar: the binary representation of the denominator will always start with "1", for the bar number will always chop it to its shortest representation. That means that the first bit of the denominator needs not be stored. In this case, the number "3" can be represented just by the binary value "1" and a bar value of "1":

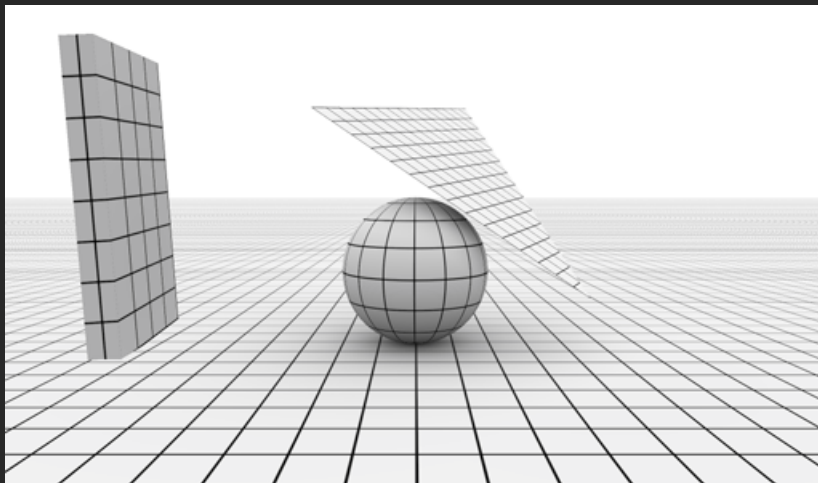
7/3 = 0 00001 000000000000000000000000111 1

This trick not only saved me one precious bit, but it also came with a beautiful side effect: when the bar is set to zero, it naturally and simultaneously implies that the denominator is 1 and that no room is needed to store it. In turn that means that my rational representation of numbers was suddenly fully compatible with regular integer representation and arithmetic as long as the numbers stayed below 2^{26} , which is a pretty decent number. Such a nice surprise! So in theory I could use the exact same data type, "rational", to perform the common rendering and shading operations but also do all the logic and control flow tasks in the path tracer - I'd no longer need to have two data types like in most renderers ("int" and "float") and convert back and forth! However the clock was ticking, so I did not change all my loop indices from "int" to "rational". The night was running short and I still had multiple things I wanted to try.

But first, let's look at what I could get using my new full 32 bit (115126) Floating Bar rational numbers:



32 bit (115|26) floating **bar** rationals



32 bit floating **point** reference

Ohhh, not bad! I still had some artifacts in the sphere, but at this point I decided I'd blame my poor square root implementation rather than the Floating Bar numbers themselves. On the other hand, the box and the triangle were really cleanly rendered.

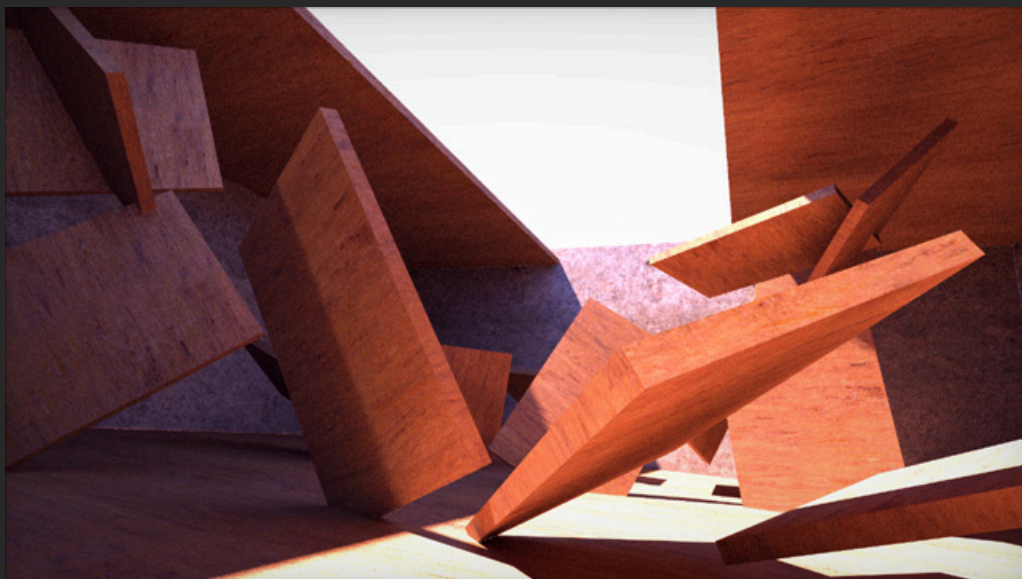
Also note that the number of exactly resolved pixels per image increased with respect to the previous versions thanks to the adaptive bit allocation technique. By allowing larger numbers to exist before reaching overflows in denominator or numerator, I increased the chances of the GCD to find common factors to reduce. And so, the floating bar not only increased the range of the representable numbers and delayed overflows driven lossy normalizations, but also got an extra kick of quality improvements and number of mathematically exact pixels!

A bigger challenge

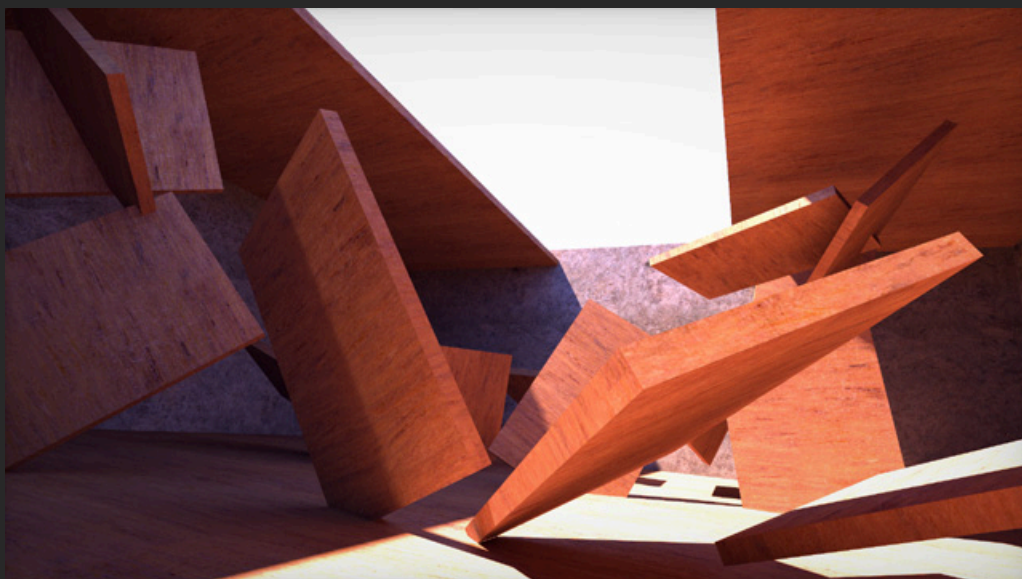
At this point I was ready to try a bigger test than my basic triangle/box/sphere scene. So I implemented a bare minimum pathtracer (not necessarily physically accurate or even physically motivated) and set a scene with some boxes and two light sources with a reference GPU implementation that you can find here:

<https://www.shadertoy.com/view/Xd2fzR>.

I converted the ray-tracing code and scene to C++ again, removed some unnecessary ray normalizations once more, and launched the render with my new Rational Bar based rational numbers. This is what I got:

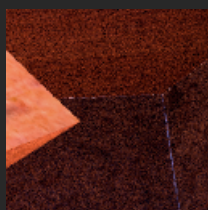
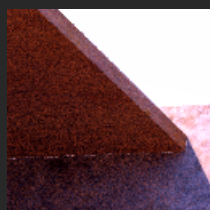
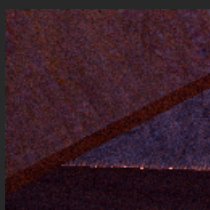


32 bit floating **bar** rationals



32 bit floating **point** reference

Okey, now this was really not bad at all! I was really excited. Although, under the magnifying glass, I could clearly see some light leaks at the corners where the edges of the floor and ceiling meet. Here are some close ups showing the errors:



These could be due to a problem in my ray-box intersection implementation that only manifested in rationals, I wouldn't be surprised. Or maybe I was simply hitting the limits of what rationals can do. Anyway, I was pretty happy. And also I had other developments or experiments I wanted to try or think about in the short time left:

Some other experiments

Exact Arithmetic in 64 bit

After the success of 32 bit (115126) Floating Bar rationals, I decided to try 64 bits and see if I could get all pixels of the screen to be exact. So I implemented 116157 rationals quickly, although first I had to learn some more cool x64 intrinsics for bit shifting and 128bit integer arithmetic. These 57 bits of numerator/denominator allowed for a much bigger range of distances to be traced. And indeed, for simple scenes with a few triangles (not the path-traced scene above), I could get **exact** results. This was a success!

However, ironically enough, the coplanarity test that I implemented in order to test the correctness required a few dot and cross products, which made the numbers start renormalizing themselves :) But I knew the render was exact because for many pixels no breakpoint was hit in the renormalization code.

Square roots

Square roots. This is a topic that I decided to stop a bit and learn new things this my second and last night of research. I wanted to implement the best possible square rooter for rationals. My current naive approach, which is **bad**, was to take the integer square root of the numerator (with proper rounding) and then do the same with the denominator. Since the square root of a fraction is the fraction of the square roots of its numerator and denominator, overall this approach returns something okeish that is not too far from the best possible answer. But it definitely does not produce the best rational approximation to the square root of a rational number. Intuitively, it is performing two approximations instead of a single one.

What I tried is the following: what we really are after here are two integers **x** and **y** such that

$$\sqrt{\frac{a}{b}} = \frac{y}{x}$$

We can square both sides and move terms around, and rewrite our problem to be that of finding the (non trivial) solution to the following diophantine equation (diophantine means that we are only interested in integer solutions):

$$ax^2 - by^2 = 0$$

After some online Wikipedia browsing I found that this particular equation is a so-called "Modified Pell's" equation, or "Pell's Like" equation. There are algorithms to find the smallest values of **x** and **y** that fulfill the equation. Unfortunately I fell for the Wikipedia rabbit hole, and didn't find the time and energy to try implementing any of it.

More efficient reduction

In my last minutes before my night and hence the experiment was over, I thought of trying to take advantage of the multiple terms that combine together in complex geometrical operators such as the cross product. For example, the first component of a cross product is

$$(s \times t)_x = \frac{acfh - bdeg}{bdfh}$$

assuming $s_y=a/b$, $t_z=c/d$, $t_y=e/f$, $s_z=g/h$

This meant that now I could try finding common factors between **a** and **d**, or **e** and **h**, for example, and use them for early reductions.

Another idea I had was that if at some point rendering speed was a concern, I could try skipping the very expensive GCD steps altogether and only implement normalization. A quick test proved that doing so still produces reasonable renders and works just fine at a much faster speed. It does come with less number of arithmetically exact results though, of course.

However, I thought of an interesting compromise (perhaps): instead of implementing a generic GCD routine (or circuitry if this was ever to be implemented in hardware), what about implementing something

mathematically simpler, hardcoded and efficient, that finds divisibility only by the numbers 2, 3 and 5, and skips 7, 11, 13 and all the larger primes. While not an exhaustive factorization, it should be able to catch a great amount of potential rational reductions in practice. After all, divisibility by 2 is three times more common than divisibility by 7, and twenty times more common than divisibility by 41! So statistically just simplifying by 2, 3 and 5 should cover a large percentage of the ground required to delay overflows? I didn't do the basic math analysis to determine this.

Conclusion

After this exercise I ended up believing that there's perhaps room for a number representation that is based on rational numbers similar to what I am here calling "Floating Bar", a representation that is compatible with integers and can do many operations in exact arithmetic for many problems. The 64 bit version (116157) can go a long way, although the 32 bit version (115126) can already produce some interesting renderings.

If this hadn't been a two nights experiment but something made at work at a studio or a company, some possible next steps would have been:

- * Get an histogram of number of exactly traced pixels vs not (or in other words, how frequently normalization happens and where)
- * Try the hardcoded 2, 3 and 5 factor reduction, and measure percentage of lost exact pixels
- * Show pixel difference between floating point rendering and float bar rendering
- * Find creative ways to express inf and nan
- * Implement the actual detection of nan, inf, underflow and overflow

Overall, this was a fun exploration. There were a few surprises on the way, a small invention, and lots of learning about Pell's equation, square roots, GCD, x86_64 intrinsics, Riemann Zeta function, and some other things. I'm pretty happy with it. I hope you too enjoyed the reading!

inigo quilez - learning computer graphics since 1994